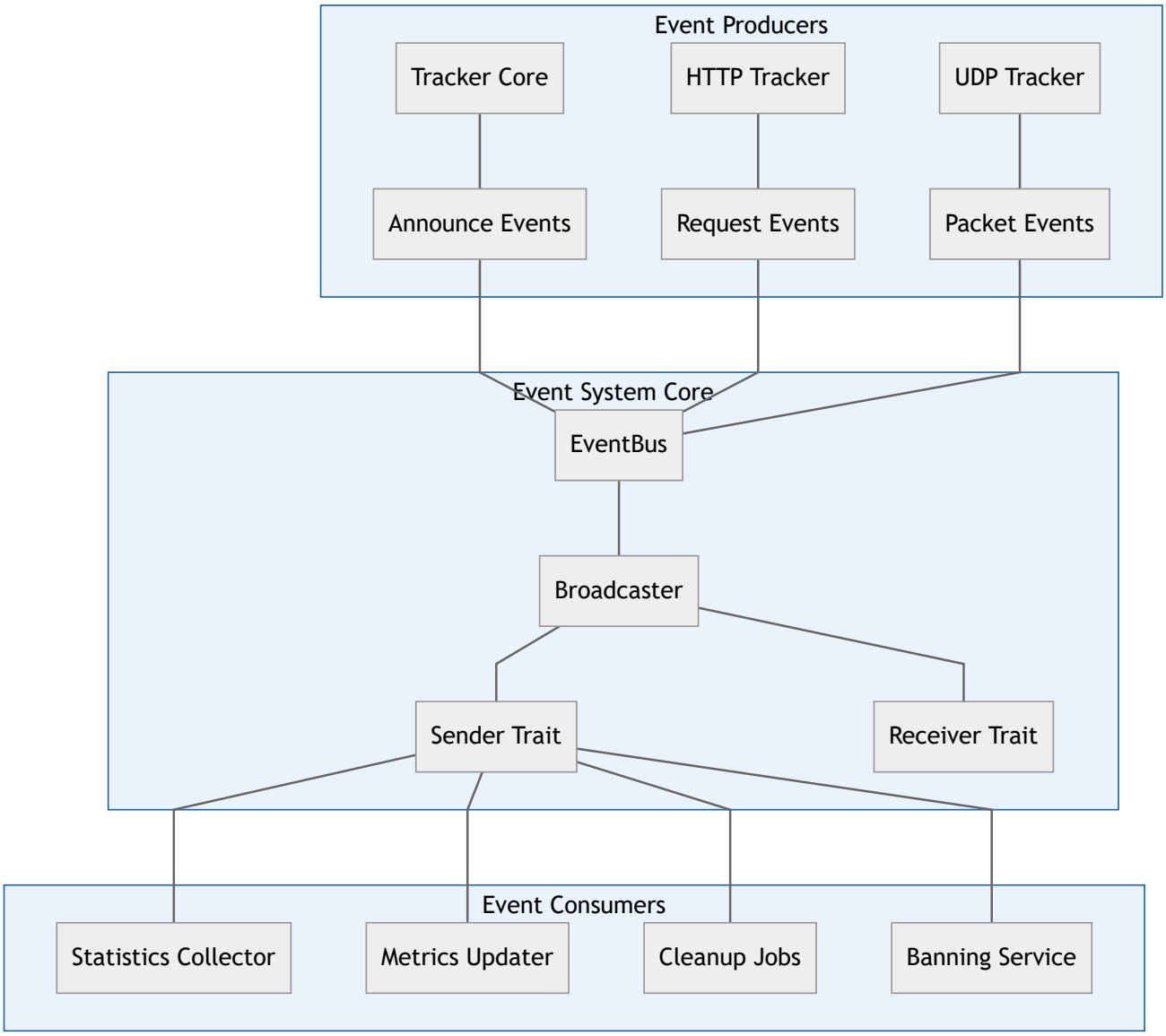
06. 이벤트 시스템 및 비동기 처리 분석

6.1 이벤트 시스템 개요

Torrust Tracker는 이벤트 기반 아키텍처를 채택하여 시스템의 확장성과 모듈성을 보장합니다. 이벤트 시스템은 `packages/events` 패키지에 구현되어 있으며, 발행-구독(Pub-Sub) 패턴을 통해 컴포넌트 간의 느슨한 결합을 제공합니다.

이벤트 시스템 아키텍처



핵심 이벤트 컴포넌트

EventBus 구조체

위치: `packages/events/src/bus.rs`

```rust
#[derive(Clone, Debug)]
pub struct EventBus<Event: Sync + Send + Clone + 'static> {
pub sender_status: SenderStatus,
```

```
pub broadcaster: Broadcaster<Event>,
}
```

역할: 이벤트 버스는 이벤트 발행자와 구독자 간의 중앙 집중식 통신 채널을 제공합니다.

주요 기능:

- 이벤트 발송자(Sender) 생성 및 관리
- 이벤트 수신자(Receiver) 생성 및 관리
- 발송자 활성화/비활성화 제어

Broadcaster 구현체

위치: `packages/events/src/broadcaster.rs`

```
#[derive(Clone, Debug)]
pub struct Broadcaster<Event: Sync + Send + Clone> {
pub(crate) sender: broadcast::Sender<Event>,
}
```

역할: Tokio의 broadcast 채널을 기반으로 한 이벤트 브로드캐스터입니다.

동작 과정:

1. 기본 채널 용량 65536으로 broadcast 채널 생성
2. 이벤트 발송 시 모든 활성 구독자에게 전달
3. 구독자 수 반환으로 전달 성공 여부 확인

6.2 Job Manager와 작업 스케줄링

JobManager 클래스 분석

위치: `src/bootstrap/jobs/manager.rs`

```
#[derive(Debug, Default)]
pub struct JobManager {
jobs: Vec<Job>,
cancellation_token: CancellationToken,
}
```

역할: 백그라운드 작업들의 생명주기를 관리하고 우아한 종료를 보장합니다.

주요 메서드:

`push()` 메서드

```
pub fn push<N: Into<String>>(&mut self, name: N, handle: JoinHandle<()>) {
self.jobs.push(Job::new(name, handle));
}
```

기능: 새로운 작업을 관리자에 등록합니다.

`wait_for_all()` 메서드

```
pub async fn wait_for_all(mut self, grace_period: Duration) {
for job in self.jobs.drain(..) {
let name = job.name.clone();
if let Ok(result) = timeout(grace_period, job.handle).await {
```

```
// 작업 완료 처리
} else {
warn!(job = %name, "Job did not complete in time");
}
}
}
```

기능: 모든 작업이 완료될 때까지 대기하며, 타임아웃 처리를 제공합니다.

작업 시작 과정

위치: `src/app.rs` 의 `start_jobs()` 함수

```
async fn start_jobs(config: &Configuration, app_container: &Arc<AppContainer>)
let mut job_manager = JobManager::new();

start_swarm_coordination_registry_event_listener(config, app_container, &mut jo
start_tracker_core_event_listener(config, app_container, &mut job_manager);
start_http_core_event_listener(config, app_container, &mut job_manager);
start_udp_core_event_listener(config, app_container, &mut job_manager);
// ... 추가 작업들
job_manager
}
```

동작 과정:

1. JobManager 인스턴스 생성
2. 각 서비스별 이벤트 리스너 시작
3. 프로토콜별 트래커 인스턴스 시작
4. 정리 및 메트릭 작업 시작
5. API 서버 시작

6.3 Tokio 기반 비동기 런타임

메인 애플리케이션 런타임

위치: `src/main.rs`

```
#[tokio::main]
async fn main() {
let (_app_container, jobs) = app::run().await;


tokio::select! {
_ = tokio::signal::ctrl_c() => {
tracing::info!("Torrust tracker shutting down ...");
jobs.cancel();
jobs.wait_for_all(Duration::from_secs(10)).await;
tracing::info!("Torrust tracker successfully shutdown.");
}
}
```

```
}
```

역할: Tokio 비동기 런타임을 초기화하고 애플리케이션의 생명주기를 관리합니다.

동작 과정:

1. `#[tokio::main]` 매크로로 비동기 런타임 초기화
2. `app::run()` 으로 애플리케이션과 작업들 시작
3. `tokio::select!` 로 CTRL+C 신호 대기
4. 종료 신호 수신 시 모든 작업 취소 및 대기

비동기 작업 스폰

UDP 트래커 작업 시작

위치: `src/bootstrap/jobs/udp_tracker.rs`

```
pub  async  fn  start_job(
udp_tracker_core_container: Arc<UdpTrackerCoreContainer>,
udp_tracker_server_container: Arc<UdpTrackerServerContainer>,
form: ServiceRegistrationForm,
) -> JoinHandle<()> {
let  server = Server::new(Spawner::new(bind_to))
.start(udp_tracker_core_container, udp_tracker_server_container, form, cookie_li
.await
.expect("it should be able to start the udp tracker");


tokio::spawn(async  move {
```

```
server.state.task.await.expect("it should be able to join to the udp tracker tas
})
}
```

특징:

- `tokio::spawn()` 을 사용한 독립적인 비동기 작업 생성
- 서버 상태 모니터링 및 에러 처리
- JoinHandle 반환으로 작업 관리 가능

HTTP API 작업 시작

위치: `src/bootstrap/jobs/tracker_apis.rs`

```
async fn start_v1(
socket: SocketAddr,
tls: Option<RustlsConfig>,
http_api_container: Arc<TrackerHttpApiCoreContainer>,
form: ServiceRegistrationForm,
access_tokens: Arc<AccessTokens>,
) -> JoinHandle<()> {
let server = ApiServer::new(Launcher::new(socket, tls))
.start(http_api_container, form, access_tokens)
.await
.expect("it should be able to start to the tracker api");

tokio::spawn(async move {
```

```
assert!(!server.state.halt_task.is_closed(), "Halt channel should be open");

server.state.task.await.expect("failed to close service");

})

}
```

특징:

- TLS 지원을 포함한 HTTP 서버 시작
- 서버 상태 검증 및 모니터링
- 우아한 종료를 위한 halt 채널 관리

6.4 이벤트 리스너들의 역할과 처리 과정

이벤트 리스너 아키텍처

시스템에는 여러 종류의 이벤트 리스너가 있으며, 각각 특정 도메인의 이벤트를 처리합니다:

1. **Swarm Coordination Registry Event Listener**
2. **Tracker Core Event Listener**
3. **HTTP Tracker Core Event Listener**
4. **UDP Tracker Core Event Listener**
5. **UDP Server Stats Event Listener**
6. **UDP Server Banning Event Listener**

Swarm Coordination Registry Event Listener

위치: `src/bootstrap/jobs/torrent_repository.rs`

```
pub fn start_event_listener(
config: &Configuration,
app_container: &Arc<AppContainer>,
cancellation_token: CancellationToken,
) -> Option<JoinHandle<()>> {
if config.core.tracker_usage_statistics {
let job = torrust_tracker_swarm_coordination_registry::statistics::event::liste
app_container.swarm_coordination_registry_container.event_bus.receiver(),
cancellation_token,
&app_container.swarm_coordination_registry_container.stats_repository,
);
Some(job)
} else {
tracing::info!("Torrent repository package event listener job is disabled.");
None
}
}
```

역할: 스웜 조정 레지스트리의 이벤트를 수신하여 통계 정보를 업데이트합니다.

처리 과정:

1. 설정에서 통계 수집이 활성화된 경우에만 시작
2. 이벤트 버스에서 수신자 생성

3. 취소 토큰을 통한 우아한 종료 지원

4. 통계 저장소에 데이터 기록

Tracker Core Event Listener

위치: `src/bootstrap/jobs/tracker_core.rs`

```
pub fn start_event_listener(
config: &Configuration,
app_container: &Arc<AppContainer>,
cancellation_token: CancellationToken,
) -> Option<JoinHandle<()>> {
if config.core.tracker_usage_statistics || config.core.tracker_policy.persisten
let job = bittorrent_tracker_core::statistics::event::listener::run_event_liste
app_container.swarm_coordination_registry_container.event_bus.receiver(),
cancellation_token,
&app_container.tracker_core_container.stats_repository,
&app_container.tracker_core_container.db_downloads_metric_repository,
app_container.tracker_core_container.core_config.tracker_policy.persistent_torre
);
Some(job)
} else {
tracing::info!("Tracker core event listener job is disabled.");
None
}
```

```
}
```

역할: 트래커 코어의 이벤트를 처리하여 통계 및 다운로드 메트릭을 관리합니다.

특징:

- 통계 수집 또는 영구 완료 통계가 활성화된 경우 시작
- 두 개의 저장소(통계, 다운로드 메트릭) 동시 관리
- 영구 토렌트 완료 통계 설정 전달

UDP Server Banning Event Listener

위치: `src/bootstrap/jobs/udp_tracker_server.rs`

```
pub fn start_banning_event_listener(
app_container: &Arc<AppContainer>,
cancellation_token: CancellationToken
) -> JoinHandle<()> {
torrust_udp_tracker_server::banning::event::listener::run_event_listener(
app_container.udp_tracker_server_container.event_bus.receiver(),
cancellation_token,
&app_container.udp_tracker_core_services.ban_service,
&app_container.udp_tracker_server_container.stats_repository,
)
}
```

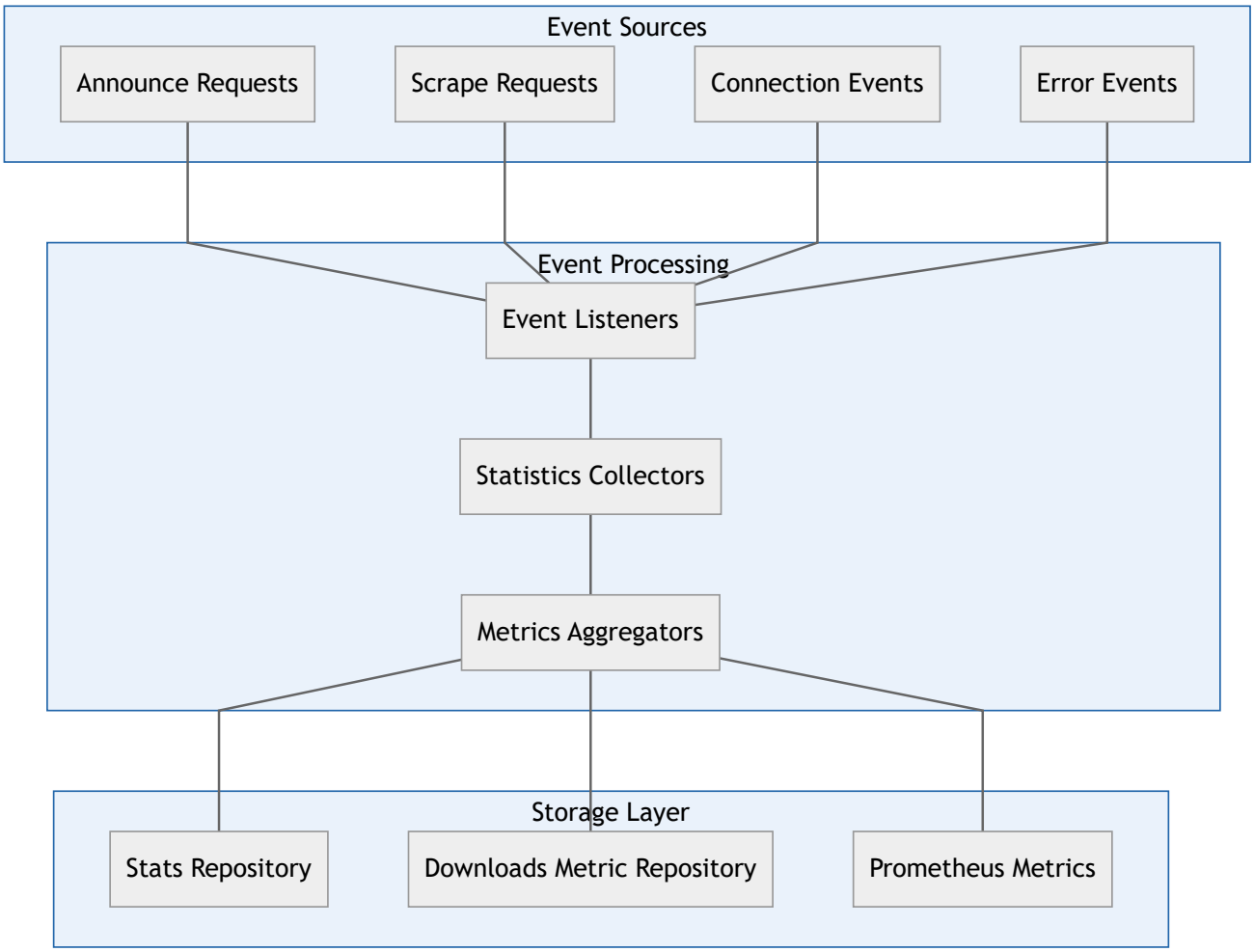
역할: UDP 서버의 차단 이벤트를 처리하여 악의적인 클라이언트를 관리합니다.

특징:

- 설정과 무관하게 항상 활성화
- 차단 서비스와 통계 저장소 연동
- 보안 관련 이벤트 처리

6.5 통계 수집 및 메트릭 시스템

통계 수집 아키텍처



Activity Metrics Updater

위치: `src/bootstrap/jobs/activity_metrics_updater.rs`

```
pub fn start_job(config: &Configuration, app_container: &Arc<AppContainer>) ->
torrust_tracker_swarm_coordination_registry::statistics::activity_metrics_update
&app_container.swarm_coordination_registry_container.swarms.clone(),
&app_container.swarm_coordination_registry_container.stats_repository.clone(),
peer_inactivity_cutoff_timestamp(config.core.tracker_policy.max_peer_timeout),
)
}


fn peer_inactivity_cutoff_timestamp(max_peer_timeout: u32) -> Duration {
CurrentClock::now_sub(&Duration::from_secs(u64::from(max_peer_timeout))).unwrap_
}
```

역할: 피어의 활동 메트릭을 주기적으로 업데이트합니다.

동작 과정:

1. 설정된 `max_peer_timeout` 을 기준으로 비활성 피어 기준 시간 계산
2. 스웜 데이터와 통계 저장소를 연동
3. 주기적으로 피어 활동 상태 업데이트

Torrent Cleanup Job

위치: `src/bootstrap/jobs/torrent_cleanup.rs`

```
pub fn start_job(config: &Core, torrents_manager: &Arc<TorrentsManager>) -> Jo
```

```
let weak_torrents_manager = std::sync::Arc::downgrade(torrents_manager);
let interval = config.inactive_peer_cleanup_interval;
let interval_in_secs = interval;


tokio::spawn(async move {
let interval = std::time::Duration::from_secs(interval);
let mut interval = tokio::time::interval(interval);
interval.tick().await;


loop {
tokio::select! {
_ = tokio::signal::ctrl_c() => {
tracing::info!("Stopping torrent cleanup job ...");
break;
}
_ = interval.tick() => {
if let Some(torrents_manager) = weak_torrents_manager.upgrade() {
let start_time = Utc::now().time();
tracing::info!("Cleaning up torrents (executed every {} secs) ...", interval_in_
torrents_manager.cleanup_torrents().await;
tracing::info!("Cleaned up torrents in: {} ms", (Utc::now().time() - start_time)
} else {
break;
```

```
}
}
}
}
})
}
```

역할: 비활성 피어와 피어가 없는 토렌트를 주기적으로 정리합니다.

동작 과정:

1. `tokio::time::interval` 로 주기적 실행 스케줄링
2. `tokio::select!` 로 CTRL+C 신호와 타이머 이벤트 동시 대기
3. Weak reference 사용으로 메모리 누수 방지
4. 실행 시간 측정 및 로깅

Prometheus 메트릭 지원

위치: `packages/metrics/src/prometheus.rs`

```
pub trait PrometheusSerializable {
fn to_prometheus(&self) -> String;
}

impl<T: PrometheusSerializable> PrometheusSerializable for &T {
fn to_prometheus(&self) -> String {
(*self).to_prometheus()
```

```
}

}
```

역할: Prometheus 형식으로 메트릭을 직렬화하는 트레이트를 제공합니다.

특징:

- 표준 Prometheus exposition 형식 지원
- 참조 타입에 대한 blanket implementation 제공
- 모니터링 시스템과의 통합 지원

6.6 백그라운드 작업 스케줄링 분석

작업 시작 순서

애플리케이션 시작 시 다음 순서로 백그라운드 작업들이 시작됩니다:

1. **이벤트 리스너들** - 각 서비스별 이벤트 처리
2. **프로토콜 인스턴스들** - UDP/HTTP 트래커 서버
3. **정리 작업들** - 토렌트 정리, 피어 비활성화 업데이트
4. **API 서버들** - REST API, Health Check API

취소 토큰 기반 종료 관리

```
impl JobManager {

pub fn cancel(&self) {

self.cancellation_token.cancel();

}
```

```
pub fn new_cancellation_token(&self) -> CancellationToken {
self.cancellation_token.clone()
}
}
```

특징:

- 모든 작업이 동일한 취소 토큰 공유
- 우아한 종료를 위한 신호 전파
- 작업별 개별 종료 처리 가능

메모리 안전성

시스템은 다음과 같은 방법으로 메모리 안전성을 보장합니다:

1. **Weak Reference 사용**: 순환 참조 방지
2. **Arc/Mutex 패턴**: 안전한 공유 상태 관리
3. **RAII 원칙**: 자동 리소스 정리
4. **취소 토큰**: 우아한 종료 보장

이러한 이벤트 시스템과 비동기 처리 아키텍처를 통해 Torrust Tracker는 높은 성능과 확장성을 제공하면서도 안정적인 운영을 보장합니다.